

Calico

Calico란?

Calico는 Tigera에서 개발한 쿠버네티스용 CNI 플러그인입니다. Flannel과 마찬가지로 서로 다른 노드에 있는 파드 간 통신 문제를 해결하기 위한 솔루션이지만, 접근 방식이 근본적으로 다릅니다. Flannel이 Overlay Network를 통해 문제를 해결하는 반면, Calico는 기본적으로 Overlay 없이, 순수 라우팅만으로 파드 간 통신을 구현하는 것을 지향합니다.

Calico는 크게 두 가지 축으로 구성됩니다.

1. **네트워킹(Networking)**: 파드 간 패킷이 실제로 오갈 수 있도록 라우팅을 구성합니다. BGP(Border Gateway Protocol) 표준 프로토콜을 그대로 가져와 사용합니다.
2. **네트워크 정책(Network Policy)**: 어떤 파드가 어떤 파드와 통신할 수 있는지를 세밀하게 제어가 가능합니다.

Flannel은 NetworkPolicy를 지원하지 않아 단순한 통신에만 적합했다면, Calico는 정교한 정책 기반 네트워크 제어가 필요한 환경, 특히 보안이 중요한 프로덕션 환경에서 널리 선택됩니다. 반대로 Flannel보다 개념적으로 복잡하고, 대규모 클러스터나 L2 세그먼트가 여러 개로 나뉜 네트워크 환경에서는 별도의 설계(Route Reflector 등)가 필요하다는 단점도 있습니다.

Calico 동작 방식

Flannel에서는 flanneld라는 하나의 프로세스가 라우팅 테이블 업데이트, 캡슐화, 디캡슐화까지 전부 처리했습니다. Calico는 이 역할을 여러 컴포넌트로 나눠서 처리합니다. 각 노드에는 다음과 같은 구성 요소가 함께 배포됩니다.

- **Felix**: 각 노드에 데몬으로 상주하는 에이전트입니다. 해당 노드에 있는 파드들의 라우팅 규칙을 커널 라우팅 테이블에 반영하고, iptables(또는 eBPF) 규칙을 만들어 NetworkPolicy를 실제로 적용합니다.
- **BIRD**: 오픈소스 BGP 라우팅 데몬입니다. Felix가 만든 로컬 라우팅 정보를 다른 노드에게 BGP로 광고(advertise)하고, 반대로 다른 노드가 광고한 라우팅 정보를 받아옵니다.
- **confd**: etcd(또는 Kubernetes API)에 저장된 설정 값을 감시하다가 변경이 생기면 BIRD의 설정 파일을 자동으로 다시 생성해주는 템플릿 엔진 역할의 컴포넌트입니다.
- **Datastore (etcd 또는 Kubernetes API)**: Calico가 사용하는 IP 풀, 노드별 할당 서브넷(블록), NetworkPolicy 같은 설정 정보가 저장되는 저장소입니다. 오늘날에는 대부분 별도 etcd 없이 Kubernetes API 서버 자체를 데이터스토어로 사용하는 방식(Kubernetes API datastore)이 기본값입니다.
- **CNI 플러그인 바이너리**: 파드가 생성될 때 kubelet에 의해 호출되어 파드에 IP를 할당하고 네트워크 인터페이스(veth pair)를 연결하는 실행 파일입니다.

Flannel과 가장 크게 다른 지점은, flannel.1 같은 가상 **TUN** 디바이스나 캡슐화 과정이 기본적으로 존재하지 않는다는 것입니다. Calico의 기본 모드에서는 노드 간에 오가는 패킷이 어떤 별도의 헤더로도 감싸지지 않은 채로 그대로 물리 네트워크를 타고 전달됩니다. 그게 어떻게 가능한지 지금부터 살펴보겠습니다.

파드가 생성될 때 무슨 일이 일어나나요?

파드가 노드에 스케줄링되면 다음과 같은 순서로 네트워크가 구성됩니다.

1. kubelet이 CNI 플러그인(calico)을 호출합니다.
2. Calico CNI 플러그인은 IPAM(IP Address Management)을 통해 파드에 할당할 IP를 하나 받아옵니다. Calico의 IPAM은 노드 단위로 미리 작은 블록(예: /26 크기의 블록)을 할당받아두고, 그 블록 안에서 파드 IP를 하나씩 나눠주는 방식으로 동작합니다.
3. 파드 안에 veth pair의 한쪽 끝을 붙이고, 반대쪽 끝은 호스트(노드)의 네트워크 네임스페이스에 남겨둡니다.
4. 호스트 쪽에 남은 veth 인터페이스로 향하는 /32 라우팅 규칙(해당 파드 IP 하나만을 위한 라우팅)을 커널 라우팅 테이블에 추가합니다. 이 작업은 Felix가 담당합니다.
5. Felix는 이 새 라우팅 정보를 BIRD에게 알려주고, BIRD는 이를 BGP를 통해 클러스터 내 다른 노드들에게 광고합니다.

즉, Flannel이 "노드 단위 서브넷 → 노드 물리 IP" 매핑을 etcd에 저장하고 flanneld가 캡슐화로 전달했다면, Calico는 "파드 IP(또는 노드가 할당받은 블록) → 다음 홉(next-hop) 노드"라는 정보를 **BGP** 라우팅 프로토콜을 통해 실시간으로 전파하고, 그 정보를 그대로 커널 라우팅 테이블에 심어둡니다.

BGP 방식 (기본 동작)

BGP는 원래 인터넷 상의 서로 다른 Autonomous System 간에 "이 IP 대역으로 가려면 나를 거쳐가라"는 라우팅 정보를 교환하기 위해 만들어진 프로토콜입니다. Calico는 이 개념을 그대로 클러스터 내부로 가져와서, 노드 하나하나를 마치 작은 라우터처럼 취급합니다.

Node A(100.96.1.1 대역을 가진 파드들이 위치)와 Node B(100.96.2.1 대역)가 있다고 가정해보겠습니다.

```
$ ip route
default via 172.20.32.1 dev eth0
100.96.1.0/26 via 100.96.1.1 dev cali1234 (로컬 파드로 향하는 라우팅)
100.96.2.0/26 via 172.20.33.103 dev eth0 proto bird
172.20.32.0/19 dev eth0 proto kernel scope link src 172.20.33.102
```

Flannel의 라우팅 테이블과 비교했을 때 가장 눈에 띄는 차이는, 목적지 대역(100.96.2.0/26)으로 가는 다음 홉이 **flannel0** 같은 가상 인터페이스가 아니라, 물리 네트워크 인터페이스인 **eth0** 그 자체라는

점입니다. `proto bird` 라는 표시는 이 라우팅 규칙이 BIRD 데몬(BGP)에 의해 동적으로 추가되었다는 것을 의미합니다.

Node A에 있는 Pod A(100.96.1.1)가 Node B에 있는 Pod B(100.96.2.2)로 패킷을 보내는 과정은 다음과 같습니다.

1. Pod A가 패킷을 보내면, 커널은 라우팅 테이블을 조회합니다. 목적지(100.96.2.2)는 `100.96.2.0/26` 규칙에 해당하고, 다음 홉은 Node B의 실제 물리 IP(172.20.33.103)로 지정되어 있습니다.
2. 커널은 이 목적지 물리 IP(172.20.33.103)에 대한 MAC 주소를 ARP로 알아내고, 원본 IP 패킷을 그대로(캡슐화 없이) eth0을 통해 내보냅니다.
3. 이 패킷은 일반적인 IP 라우팅과 동일하게 물리 네트워크(스위치, 라우터)를 거쳐 Node B에 도착합니다.
4. Node B의 커널은 패킷의 목적지(100.96.2.2)가 로컬에 있는 파드라는 것을 자신의 라우팅 테이블 (`100.96.2.0/26 via ... dev cali5678`)을 통해 알고, veth pair를 거쳐 Pod B로 전달합니다.

여기서 중요한 점은 Node A의 관점에서 Pod B의 IP(100.96.2.2)는 물리 네트워크 상의 목적지 IP와 동일하게 취급된다는 것입니다. flannel처럼 패킷을 열어보거나, 감싸거나, 사용자 공간으로 끌어올리는 과정이 전혀 없습니다. 커널 라우팅 테이블만으로 패킷 전달이 끝나기 때문에, 이 방식은 사실 Flannel의 host-gw 백엔드와 원리 자체는 동일합니다. 차이는 그 라우팅 테이블을 채우는 방법이, host-gw는 etcd를 감시하는 방식인데 반해 Calico는 표준 **BGP** 프로토콜을 사용한다는 것입니다.

왜 flannel0 같은 가상 인터페이스가 필요 없나요?

Flannel에서 flannel0(TUN 디바이스)이 필요했던 이유는, 목적지 Pod IP가 물리 네트워크 장비들이 알 수 없는 "가상의" 주소였고, 이를 실제 물리 IP로 감싸주는 캡슐화 작업이 사용자 공간의 flanneld에서 일어나야 했기 때문입니다.

반면 Calico의 BGP 모드에서는 파드의 IP 대역 자체가 물리 네트워크의 라우터들에게도 정상적인 라우팅 대상으로 알려집니다. 즉, "이 파드 IP 대역으로 가는 패킷은 이 노드로 보내면 된다"는 정보가 BGP를 통해 클러스터 내부에 광고되기 때문에, 굳이 패킷을 감쌀 필요가 없습니다.

Full-Mesh와 Route Reflector

기본적으로 Calico의 각 노드는 클러스터 내의 모든 다른 노드와 **1:1로 BGP 세션을 맺습니다(Full Mesh)**. 노드가 N개라면 노드 하나당 N-1개의 BGP 세션을 유지해야 하므로, 세션 수는 전체적으로 $N \times (N-1) / 2$ 개가 됩니다.

문제는 노드 수가 많아질수록(대략 100대 이상) 이 방식이 비효율적이 된다는 점입니다. 노드가 하나 추가 될 때마다 기존의 모든 노드가 새로운 BGP 세션을 맺어야 하고, 라우팅 정보(라우트) 하나가 변경될 때마다 모든 노드가 그 변경 사항을 전파받아야 합니다.

이를 해결하기 위한 것이 **Route Reflector(RR)** 패턴입니다. 일반 BGP에서 쓰이는 개념을 그대로 가져온 것으로, 몇 개의 노드를 Route Reflector로 지정하면, 나머지 노드들은 서로 직접 BGP 세션을 맺지 않고 오직 Route Reflector와만 세션을 맺습니다. Route Reflector가 받은 라우팅 정보를 다른 모든 노드에게 대신 중계해주는 방식입니다. 이렇게 하면 BGP 세션 수가 노드 수에 비례해서 선형적으로만 증가하게 되어, 대규모 클러스터에서도 확장성을 유지할 수 있습니다.

IP-in-IP 방식 (Cross-Subnet일 때)

앞서 설명한 순수 BGP 라우팅 방식이 항상 그대로 동작하는 것은 아닙니다. 이 방식이 성립하려면 한 가지 전제 조건이 있는데, **Node A와 Node B가 물리적으로 서로의 IP를 라우팅 없이(또는 알려진 라우팅으로) 직접 주고받을 수 있는 L2/L3 환경이어야** 한다는 것입니다.

그런데 클라우드 환경(AWS, GCP 등)에서는 흔히 각 가용 영역(AZ)이나 서브넷이 서로 다른 L2 세그먼트로 분리되어 있고, 클라우드 제공업체의 네트워크가 사용자가 마음대로 커스텀 라우팅 규칙을 주입하는 것을 허용하지 않는 경우가 많습니다. 즉, "이 파드 대역은 이 노드로 보내라"는 BGP 라우팅 정보를 물리 네트워크(클라우드의 VPC 라우터)가 그대로 받아들여주지 않는 상황이 생깁니다.

이런 환경을 위해 Calico는 **IP-in-IP(IPIP)** 캡슐화 모드를 제공합니다. 동작 방식은 이름 그대로, 원본 IP 패킷 전체를 또 다른 IP 패킷으로 한 번 더 감싸는 것입니다.

```
+-----+
| Outer IP Header (src: NodeA물리IP, dst: NodeB물리IP) |
+-----+
| Inner IP Header (src: PodA IP, dst: PodB IP)          |
| + Payload                                              |
+-----+
```

Flannel의 UDP 캡슐화와 비슷해 보이지만 중요한 차이가 있습니다. Flannel의 UDP 방식은 캡슐화 작업이 **사용자 공간의 flanneld 프로세스**에서 이루어졌지만, Calico의 IPIP는 리눅스 커널에 내장된 `ipip` 터널 모듈을 사용합니다. 즉, 캡슐화와 디캡슐화가 커널 내부에서 전부 처리되며 사용자 공간으로 패킷이 올라올 필요가 없습니다.

Calico는 이 IPIP 캡슐화를 항상 켜두는 것이 아니라, 기본 설정(`CrossSubnet` 모드)에서는 같은 서브넷(같은 L2 세그먼트)에 있는 노드끼리는 캡슐화 없이 순수 라우팅으로, 서로 다른 서브넷에 있는 노드끼리만 IPIP로 캡슐화하도록 지능적으로 선택합니다. 즉 클러스터 상황에 따라 두 방식을 혼용할 수 있습니다.

VXLAN 방식

IPIP 외에 Calico가 지원하는 또 다른 캡슐화 방식이 **VXLAN**입니다. Flannel의 VXLAN 백엔드와 원리는 거의 동일합니다. `vxlan.calico` 라는 이름의 VXLAN 디바이스가 각 노드에 생성되고, 라우팅 테이블·ARP·FDB 항목이 등록되어 커널 내부에서 캡슐화/디캡슐화가 처리됩니다.

IPIP와 VXLAN 중 어느 것을 선택해야 하는지에 대해서는 보통 다음과 같이 구분합니다.

- **IPIP**: 오버헤드가 VXLAN보다 약간 더 작습니다(IP 헤더만 추가되므로). 다만 일부 클라우드 환경이나 방화벽에서 IPIP 프로토콜(프로토콜 번호 4) 자체를 차단하는 경우가 있습니다.
- **VXLAN**: UDP 위에서 동작하기 때문에(UDP 포트 4789 사용) 대부분의 방화벽·로드밸런서·클라우드 네트워크 환경에서 더 무리 없이 통과됩니다. 또한 BGP 없이도 동작할 수 있어(VXLAN 모드에서는 BGP 대신 각 노드가 Kubernetes API를 통해 직접 서로의 서브넷 정보를 주고받음) 방화벽으로 BGP 포트(179번)가 막혀 있는 클라우드 관리형 환경(예: 관리형 쿠버네티스 서비스)에서 특히 많이 사용됩니다.

캡슐화를 권장하지 않는 이유 (성능)

Flannel의 UDP 방식이 권장되지 않았던 이유가 사용자 공간과 커널 공간 사이의 Context Switching이 있다면, Calico에서 캡슐화 모드(IPIP, VXLAN)를 피해야 하는 이유는 조금 다릅니다. 앞서 말했듯 Calico의 IPIP/VXLAN은 캡슐화 자체가 커널 안에서 일어나기 때문에 Flannel UDP 방식만큼 극단적인 오버헤드는 없습니다. 그래도 순수 BGP 라우팅(캡슐화 없음)과 비교하면 다음과 같은 차이가 발생합니다.

1. **MTU 오버헤드**: 원본 패킷을 한 번 더 감싸면 헤더가 추가되므로(IPIP는 약 20바이트, VXLAN은 약 50바이트), 그만큼 실제로 실어 나를 수 있는 payload 크기가 줄어듭니다. 이를 그대로 두면 패킷이 물리 네트워크의 MTU(보통 1500바이트)를 넘어서는 순간 단편화(fragmentation)가 발생하거나, Path MTU Discovery 과정에서 재전송이 필요해져 성능이 저하됩니다. 이를 막기 위해 캡슐화를 사용할 때는 파드 네트워크 인터페이스의 MTU를 오버헤드만큼 미리 줄여서 설정해주어야 합니다.
2. **CPU 오버헤드**: 커널 안에서 처리된다고 해도, 매 패킷마다 헤더를 추가/제거하는 연산과 체크섬 재계산이 필요합니다. 트래픽이 많을수록 이 연산 비용이 누적됩니다.
3. **하드웨어 오프로드 및 가시성 저하**: 캡슐화되지 않은 순수 IP 패킷은 클라우드 제공업체의 네트워크 인터페이스 카드(NIC)가 제공하는 오프로드 기능(체크섬 오프로드, GRO/GSO 등)이나 기존 방화벽·모니터링 장비의 혜택을 온전히 받을 수 있지만, 캡슐화된 패킷은 이런 장비들이 겉으로 보이는 Outer 헤더만 보고 판단하게 되어 최적화나 트래픽 분석의 정밀도가 떨어질 수 있습니다.

이런 이유로 Calico를 운영할 때는 가능하다면 **캡슐화 없는 순수 BGP 라우팅 모드**를 사용하는 것이 가장 권장되며, 클라우드 환경의 제약으로 어쩔 수 없이 캡슐화가 필요한 경우에도 전체 노드가 아니라 서로 다른 서브넷 간에만(CrossSubnet) 캡슐화가 적용되도록 구성하는 것이 일반적인 모범 사례입니다.

NetworkPolicy 지원 (Calico의 핵심 강점)

지금까지 설명한 BGP·라우팅·캡슐화는 패킷이 노드 사이를 어떻게 이동하는가에 대해 보았습니다. 하지만 앞서 언급했듯, Calico가 널리 사용되는 이유중 하나는 NetworkPolicy 구현에 있습니다.

Flannel은 이 기능을 아예 지원하지 않기 때문에, Flannel만 설치된 클러스터에서 NetworkPolicy 리소스를 만들어도 실제로는 아무 효과가 없습니다. 반면 Calico의 Felix는 각 노드에서 다음과 같은 작업을 수행합니다.

1. Kubernetes API(혹은 etcd)에 정의된 **NetworkPolicy** / **GlobalNetworkPolicy** (Calico 자체 확장 리소스) 객체를 지속적으로 감시합니다.
2. 정책에 정의된 규칙(어떤 라벨을 가진 파드로부터/향하는 트래픽을 허용 또는 차단할지)을 해당 노드에 있는 파드들에 대해 실제 방화벽 규칙으로 변환합니다.
3. 변환된 규칙을 커널의 **iptables**(또는 최신 버전의 **eBPF 데이터플레인**)에 반영합니다.

즉, 패킷이 veth 인터페이스를 통해 노드로 들어오고 나갈 때, 커널은 라우팅 테이블로 어디로 보낼지를 결정하기 전후에 iptables 체인(또는 eBPF 프로그램)을 거치면서 "이 패킷이 허용되는지"를 검사받습니다. 이 검사 지점에서 정책에 위배되는 트래픽은 그대로 드롭됩니다.

전통적인 iptables 방식은 규칙 수가 많아지면(파드·정책 수가 늘어날수록) 규칙을 순차적으로 탐색하는데 드는 비용이 커지는 단점이 있습니다. 이 때문에 Calico는 커널 4.x 이상에서 **eBPF 데이터플레인** 모드를 지원하는데, 이 모드에서는 규칙이 순차 탐색이 아니라 해시 기반 조회로 처리되어 대규모 클러스터에서도 정책 적용에 따른 지연이 거의 발생하지 않습니다. 더 나아가 eBPF 모드에서는 kube-proxy가 하던 서비스(ClusterIP) 로드밸런싱 기능까지 대체할 수 있어, kube-proxy 없이도 동작하는 구성이 가능합니다.

정리

	Flannel	Calico (기본: BGP)
기본 통신 방식	Overlay Network (UDP/VXLAN 캡슐화)	캡슐화 없는 순수 라우팅 (필요시 IPIP/VXLAN)
라우팅 정보 전파	etcd를 각 flanneld가 감시(watch)	BGP 프로토콜로 노드 간 직접 교환
핵심 데몬	flanneld (단일 프로세스)	Felix + BIRD + confd (역할 분리)
NetworkPolicy	미지원	지원 (iptables 또는 eBPF)
대표 성능 이슈	UDP 모드에서 User/Kernel 공간 전환 오버헤드	캡슐화(IPIP/VXLAN) 모드에서의 MTU/CPU 오버헤드
대규모 확장 시 고려사항	특별한 구조 변경 불필요	BGP Full-Mesh 한계 → Route Reflector 필요

정리하면, Calico는 파드가 생성될 때 CNI 플러그인과 IPAM을 통해 IP를 할당받고, Felix가 이를 노드의 라우팅 테이블에 반영하면 BIRD가 이 정보를 BGP로 클러스터 전체에 전파하는 방식으로 동작합니다. 같은 L2 세그먼트 안에서는 캡슐화 없이 순수 라우팅으로 패킷이 오가고, 서로 다른 서브넷 사이에서만 필요에 따라 IPIP나 VXLAN으로 캡슐화됩니다. 그리고 이 네트워킹 기능과는 별개로, Felix가 NetworkPolicy를 iptables 또는 eBPF 규칙으로 변환해 실제로 적용함으로써 Flannel에는 없는 세밀한 트래픽 제어 기능을 제공하는 것이 Calico의 핵심입니다.